| Ex. No. 7 | PL/SQL - TRIGGERS |
|---|---|
| Date of Exercise | 09/09/2026 |

**Aim**

To solve the given problems using triggers.

**Description**

A trigger is procedural code that runs automatically in response to a specified event -- an INSERT, UPDATE or DELETE -- on a particular table, mainly to enforce the integrity of the data in a database. Unlike a procedure, which is explicitly invoked by a user, application or another trigger, a trigger fires implicitly whenever its triggering event occurs. BEFORE triggers run ahead of the triggering statement and can modify the row about to be written or reject it outright, while AFTER triggers run once the statement has completed and are typically used to cascade changes to other tables or log what happened.

**1. Whenever there is a new conference event is created, make sure it is conducted in venue id 5.**

**Sample Code:**

```
CREATE OR REPLACE FUNCTION URK24CS1021_conf_venue_fn() RETURNS TRIGGER
LANGUAGE plpgsql AS $$
BEGIN
    IF NEW.name ILIKE '%Conference%' THEN
        NEW.venueid := 5;
    END IF;
    RETURN NEW;
END;
$$;
CREATE TRIGGER URK24CS1021_t1_conf_venue
BEFORE INSERT ON URK24CS1021_event
FOR EACH ROW EXECUTE FUNCTION URK24CS1021_conf_venue_fn();
INSERT INTO URK24CS1021_event (eventid, name, eventdate, eventtime, venueid,
description)
VALUES (4, 'AI Conference', '2023-10-10', '2023-10-10 09:00:00', 2, 'Annual AI
conference');
SELECT eventid, name, venueid FROM URK24CS1021_event WHERE eventid = 4;
```

**Sample Output:**

```
psql — labdb

labdb=# CREATE OR REPLACE FUNCTION URK24CS1021_conf_venue_fn() RETURNS TRIGGER
labdb(# LANGUAGE plpgsql AS $$
labdb(# BEGIN
labdb(#     IF NEW.name ILIKE '%Conference%' THEN
labdb(#         NEW.venueid := 5;
labdb(#     END IF;
labdb(#     RETURN NEW;
labdb(# END;
labdb(# $$;
CREATE FUNCTION

labdb=# CREATE TRIGGER URK24CS1021_t1_conf_venue
labdb(# BEFORE INSERT ON URK24CS1021_event
labdb(# FOR EACH ROW EXECUTE FUNCTION URK24CS1021_conf_venue_fn();
CREATE TRIGGER

labdb=# INSERT INTO URK24CS1021_event (eventid, name, eventdate, eventtime, venueid, description)
labdb(# VALUES (4, 'AI Conference', '2023-10-10', '2023-10-10 09:00:00', 2, 'Annual AI conference');
INSERT 0 1

labdb=# SELECT eventid, name, venueid FROM URK24CS1021_event WHERE eventid = 4;
 eventid |     name      | venueid
---------+---------------+---------
       4 | AI Conference |       5
(1 row)

labdb=#
```

**2. Create a trigger to implement referential integrity policy "on delete set null" in the event table. That is when venue id is deleted from the venue table, set null for deleted venue in the event table.**

**Sample Code:**

```
CREATE OR REPLACE FUNCTION URK24CS1021_venue_del_fn() RETURNS TRIGGER
LANGUAGE plpgsql AS $$
BEGIN
    UPDATE URK24CS1021_event SET venueid = NULL WHERE venueid = OLD.venueid;
    RETURN OLD;
END;
$$;
CREATE TRIGGER URK24CS1021_t2_venue_del
BEFORE DELETE ON URK24CS1021_location
FOR EACH ROW EXECUTE FUNCTION URK24CS1021_venue_del_fn();
DELETE FROM URK24CS1021_location WHERE venueid = 3;
SELECT eventid, name, venueid FROM URK24CS1021_event WHERE eventid = 1;
```

**Sample Output:**

```
psql — labdb

labdb=# CREATE OR REPLACE FUNCTION URK24CS1021_venue_del_fn() RETURNS TRIGGER
labdb(# LANGUAGE plpgsql AS $$
labdb(# BEGIN
labdb(#     UPDATE URK24CS1021_event SET venueid = NULL WHERE venueid = OLD.venueid;
labdb(#     RETURN OLD;
labdb(# END;
labdb(# $$;
CREATE FUNCTION

labdb=# CREATE TRIGGER URK24CS1021_t2_venue_del
labdb(# BEFORE DELETE ON URK24CS1021_location
labdb(# FOR EACH ROW EXECUTE FUNCTION URK24CS1021_venue_del_fn();
CREATE TRIGGER

labdb=# DELETE FROM URK24CS1021_location WHERE venueid = 3;
DELETE 1

labdb=# SELECT eventid, name, venueid FROM URK24CS1021_event WHERE eventid = 1;
 eventid |    name    | venueid
---------+------------+---------
       1 | Jazz Night |
(1 row)

labdb=#
```

**3. Create a trigger that prints the change in venue every time venue of an event is changed.**

**Sample Code:**

```
CREATE OR REPLACE FUNCTION URK24CS1021_venue_change_fn() RETURNS TRIGGER
LANGUAGE plpgsql AS $$
BEGIN
    RAISE NOTICE 'Event % venue changed from % to %', OLD.eventid, OLD.venueid,
NEW.venueid;
    RETURN NEW;
END;
$$;
CREATE TRIGGER URK24CS1021_t3_venue_change
AFTER UPDATE OF venueid ON URK24CS1021_event
FOR EACH ROW EXECUTE FUNCTION URK24CS1021_venue_change_fn();
UPDATE URK24CS1021_event SET venueid = 4 WHERE eventid = 2;
```

**Sample Output:**

```
psql — labdb

labdb=# CREATE OR REPLACE FUNCTION URK24CS1021_venue_change_fn() RETURNS TRIGGER
labdb(# LANGUAGE plpgsql AS $$
labdb(# BEGIN
labdb(#     RAISE NOTICE 'Event % venue changed from % to %', OLD.eventid, OLD.venueid, NEW.venueid;
labdb(#     RETURN NEW;
labdb(# END;
labdb(# $$;
CREATE FUNCTION

labdb=# CREATE TRIGGER URK24CS1021_t3_venue_change
labdb(# AFTER UPDATE OF venueid ON URK24CS1021_event
labdb(# FOR EACH ROW EXECUTE FUNCTION URK24CS1021_venue_change_fn();
CREATE TRIGGER

labdb=# UPDATE URK24CS1021_event SET venueid = 4 WHERE eventid = 2;
UPDATE 1
NOTICE:  Event 2 venue changed from 2 to 4

labdb=#
```

**4. Write a trigger to stop the events happening in venue "Grand Theater".**

**Sample Code:**

```
CREATE OR REPLACE FUNCTION URK24CS1021_block_grand_fn() RETURNS TRIGGER
LANGUAGE plpgsql AS $$
BEGIN
    IF NEW.venueid = (SELECT venueid FROM URK24CS1021_location WHERE name =
'Grand Theater') THEN
        RAISE EXCEPTION 'Events are not allowed at Grand Theater';
    END IF;
    RETURN NEW;
END;
$$;
CREATE TRIGGER URK24CS1021_t4_block_grand
BEFORE INSERT ON URK24CS1021_event
FOR EACH ROW EXECUTE FUNCTION URK24CS1021_block_grand_fn();
INSERT INTO URK24CS1021_event (eventid, name, eventdate, eventtime, venueid,
description)
VALUES (5, 'Community Play', '2023-11-01', '2023-11-01 19:00:00', 5, 'Local
theater play');
```

**Sample Output:**

```
psql — labdb

labdb=# CREATE OR REPLACE FUNCTION URK24CS1021_block_grand_fn() RETURNS TRIGGER
labdb(# LANGUAGE plpgsql AS $$
labdb(# BEGIN
labdb(#     IF NEW.venueid = (SELECT venueid FROM URK24CS1021_location WHERE name = 'Grand Theater') THEN
labdb(#         RAISE EXCEPTION 'Events are not allowed at Grand Theater';
labdb(#     END IF;
labdb(#     RETURN NEW;
labdb(# END;
labdb(# $$;
CREATE FUNCTION

labdb=# CREATE TRIGGER URK24CS1021_t4_block_grand
labdb(# BEFORE INSERT ON URK24CS1021_event
labdb(# FOR EACH ROW EXECUTE FUNCTION URK24CS1021_block_grand_fn();
CREATE TRIGGER

labdb=# INSERT INTO URK24CS1021_event (eventid, name, eventdate, eventtime, venueid, description)
labdb(# VALUES (5, 'Community Play', '2023-11-01', '2023-11-01 19:00:00', 5, 'Local theater play');
ERROR:  Events are not allowed at Grand Theater
CONTEXT:  PL/pgSQL function urk24cs1021_block_grand_fn() line 4 at RAISE

labdb=#
```

**5. Create a trigger for ticket table that inserts the newly inserted values in the ticket_audit table in case of insert and inserts old existing values in ticket_audit table in case of a change on status 'confirmed' from 'reserved'. The structure of the ticket_audit table is same as ticket table.**

**Sample Code:**

```
CREATE TABLE URK24CS1021_ticket_audit (
    ticketid numeric(10), eventid numeric(10), userid numeric(10),
    price numeric(10,2), status varchar(50), barcode varchar(50)
);
CREATE OR REPLACE FUNCTION URK24CS1021_ticket_audit_fn() RETURNS TRIGGER
LANGUAGE plpgsql AS $$
BEGIN
    IF TG_OP = 'INSERT' THEN
        INSERT INTO URK24CS1021_ticket_audit
        VALUES (NEW.ticketid, NEW.eventid, NEW.userid, NEW.price, NEW.status,
NEW.barcode);
    ELSIF TG_OP = 'UPDATE' AND OLD.status = 'reserved' AND NEW.status =
'confirmed' THEN
        INSERT INTO URK24CS1021_ticket_audit
        VALUES (OLD.ticketid, OLD.eventid, OLD.userid, OLD.price, OLD.status,
OLD.barcode);
    END IF;
    RETURN NEW;
END;
$$;
CREATE TRIGGER URK24CS1021_t5_ticket_audit
AFTER INSERT OR UPDATE ON URK24CS1021_ticket
FOR EACH ROW EXECUTE FUNCTION URK24CS1021_ticket_audit_fn();
INSERT INTO URK24CS1021_ticket (ticketid, eventid, userid, price, status,
barcode)
VALUES (3, 3, 103, 80.00, 'reserved', 'BC7003');
```

```
UPDATE URK24CS1021_ticket SET status = 'confirmed' WHERE ticketid = 1;
SELECT * FROM URK24CS1021_ticket_audit;
```

**Sample Output:**

```
psql — labdb

labdb=# CREATE TABLE URK24CS1021_ticket_audit (
labdb(#     ticketid numeric(10), eventid numeric(10), userid numeric(10),
labdb(#     price numeric(10,2), status varchar(50), barcode varchar(50)
labdb(# );
CREATE TABLE

labdb=# CREATE OR REPLACE FUNCTION URK24CS1021_ticket_audit_fn() RETURNS TRIGGER
labdb(# LANGUAGE plpgsql AS $$
labdb(# BEGIN
labdb(#     IF TG_OP = 'INSERT' THEN
labdb(#         INSERT INTO URK24CS1021_ticket_audit
labdb(#         VALUES (NEW.ticketid, NEW.eventid, NEW.userid, NEW.price, NEW.status, NEW.barcode);
labdb(#     ELSIF TG_OP = 'UPDATE' AND OLD.status = 'reserved' AND NEW.status = 'confirmed' THEN
labdb(#         INSERT INTO URK24CS1021_ticket_audit
labdb(#         VALUES (OLD.ticketid, OLD.eventid, OLD.userid, OLD.price, OLD.status, OLD.barcode);
labdb(#     END IF;
labdb(#     RETURN NEW;
labdb(# END;
labdb(# $$;
CREATE FUNCTION

labdb=# CREATE TRIGGER URK24CS1021_t5_ticket_audit
labdb(# AFTER INSERT OR UPDATE ON URK24CS1021_ticket
labdb(# FOR EACH ROW EXECUTE FUNCTION URK24CS1021_ticket_audit_fn();
CREATE TRIGGER

labdb=# INSERT INTO URK24CS1021_ticket (ticketid, eventid, userid, price, status, barcode)
labdb(# VALUES (3, 3, 103, 80.00, 'reserved', 'BC7003');
INSERT 0 1

labdb=# UPDATE URK24CS1021_ticket SET status = 'confirmed' WHERE ticketid = 1;
UPDATE 1

labdb=# SELECT * FROM URK24CS1021_ticket_audit;
 ticketid | eventid | userid | price  |  status  | barcode
----------+---------+--------+--------+----------+---------
        3 |       3 |    103 |  80.00 | reserved | BC7003
        1 |       1 |    101 | 100.00 | reserved | BC7001
(2 rows)

labdb=#
```

**6. Disable the above created trigger.**

**Sample Code:**

```
ALTER TABLE URK24CS1021_ticket DISABLE TRIGGER URK24CS1021_t5_ticket_audit;
INSERT INTO URK24CS1021_ticket (ticketid, eventid, userid, price, status,
barcode)
VALUES (4, 2, 102, 60.00, 'confirmed', 'BC7004');
SELECT * FROM URK24CS1021_ticket_audit;
```

**Sample Output:**

```
psql — labdb

labdb=# ALTER TABLE URK24CS1021_ticket DISABLE TRIGGER URK24CS1021_t5_ticket_audit;
ALTER TABLE

labdb=# INSERT INTO URK24CS1021_ticket (ticketid, eventid, userid, price, status, barcode)
labdb(# VALUES (4, 2, 102, 60.00, 'confirmed', 'BC7004');
INSERT 0 1

labdb=# SELECT * FROM URK24CS1021_ticket_audit;
 ticketid | eventid | userid | price  |  status   | barcode
----------+---------+--------+--------+-----------+---------
        3 |       3 |    103 |  80.00 | reserved  | BC7003
        1 |       1 |    101 | 100.00 | reserved  | BC7001
(2 rows)

labdb=#
```

**7. Drop the first created trigger.**

**Sample Code:**

```
DROP TRIGGER URK24CS1021_t1_conf_venue ON URK24CS1021_event;
INSERT INTO URK24CS1021_event (eventid, name, eventdate, eventtime, venueid,
description)
VALUES (6, 'Robotics Conference', '2023-12-01', '2023-12-01 10:00:00', 2,
'Robotics expo');
SELECT eventid, name, venueid FROM URK24CS1021_event WHERE eventid = 6;
```

**Sample Output:**

```
psql — labdb

labdb=# DROP TRIGGER URK24CS1021_t1_conf_venue ON URK24CS1021_event;
DROP TRIGGER

labdb=# INSERT INTO URK24CS1021_event (eventid, name, eventdate, eventtime, venueid, description)
labdb(# VALUES (6, 'Robotics Conference', '2023-12-01', '2023-12-01 10:00:00', 2, 'Robotics expo');
INSERT 0 1

labdb=# SELECT eventid, name, venueid FROM URK24CS1021_event WHERE eventid = 6;
 eventid |        name         | venueid
---------+---------------------+---------
       6 | Robotics Conference |       2
(1 row)

labdb=#
```

**RESULT**

The SQL queries were executed and the desired output was obtained.